BÁO CÁO THIẾT KẾ KHO DỮ LIỆU, ETL VÀ OLAP

Đề tài: E-Commerce Olist (Brazil)

Môn học: Kho dữ liệu và Hệ hỗ trợ Quyết định (CO4031)

Sinh viên thực hiện

Ngày 28 tháng 10 năm 2026

Mục lục

1 Thiết kế Mô hình Kho Dữ liệu (Data Warehouse Modeling)

1.1 Phương pháp luận Ralph Kimball và Quy trình 4 bước

Trong đề tài này, mô hình Kho dữ liệu (Data Warehouse - DW) được xây dựng tuân thủ nghiêm ngặt theo phương pháp luận thiết kế đa chiều chuẩn của Ralph Kimball (Kimball Methodology). Quy trình thiết kế được cụ thể hóa qua 4 bước:

1. **Chọn quy trình nghiệp vụ (Select the Business Process):** Nghiệp vụ cốt lõi được lựa chọn là *Quản lý Đơn hàng và Thực hiện Đơn hàng (Order Fulfillment & Sales)* của sàn thương mại điện tử Olist. Quy trình này phản ánh vòng đời từ lúc người mua đặt hàng, người bán chuẩn bị hàng, vận chuyển đến tay người mua và khách hàng đánh giá chất lượng dịch vụ.

2. **Xác định hạt dữ liệu (Declare the Grain):** Hạt dữ liệu (Grain) là yếu tố quyết định mức độ chi tiết của kho dữ liệu. Nhóm quyết định chọn hạt dữ liệu ở mức độ nguyên tử chi tiết nhất (Atomic Grain): **Từng mặt hàng trong một đơn hàng** (*Line-item level*, tương ứng với cặp định danh `order_id` và `order_item_id`). Việc chọn mức hạt chi tiết nhất cho phép hệ thống thực hiện mọi thao tác khoan sâu (Drill-down) mà không bị mất mát thông tin.

3. **Xác định các chiều dữ liệu (Identify the Dimensions):** Dựa trên quy trình nghiệp vụ và mức hạt đã chọn, 4 chiều phân tích (Dimension Tables) được thiết lập bao gồm:

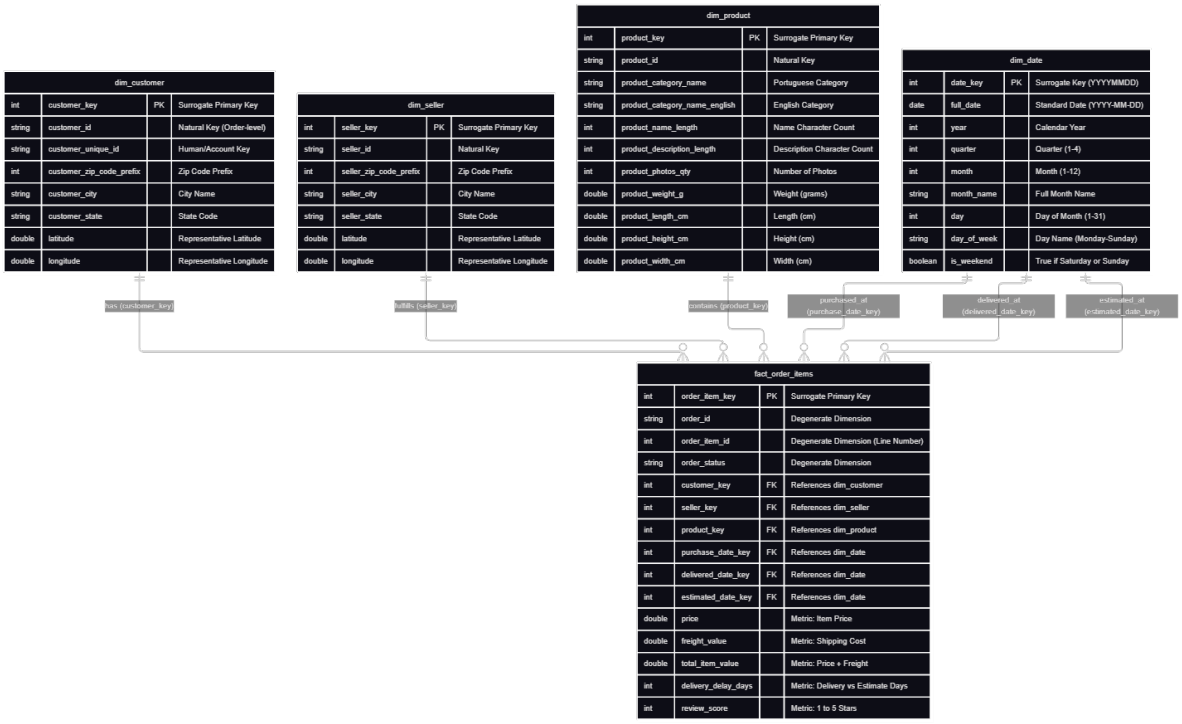
   - **Chiều Khách hàng (`dim_customer`):** Lưu trữ định danh, khu vực địa lý, mã vùng (Zip Code), thành phố, bang và tọa độ địa lý đại diện (Latitude, Longitude).
   - **Chiều Người bán (`dim_seller`):** Chứa thông tin về đối tác cung cấp hàng hóa, vị trí kho bãi, thành phố, bang và tọa độ.
   - **Chiều Sản phẩm (`dim_product`):** Phân loại hàng hóa, tên danh mục chuẩn hóa (tiếng Anh và Bồ Đào Nha), kích thước đóng gói (dài, rộng, cao) và khối lượng.
   - **Chiều Thời gian (`dim_date`):** Chiều đóng vai trò chuyển vùng (Role-Playing Dimension) cho nhiều mốc thời gian: Ngày đặt hàng (*purchase*), Ngày giao hàng thực tế (*delivered*), Ngày giao hàng dự kiến (*estimated*). Chiều này phân rã thời gian thành: ngày, tháng, quý, năm, thứ trong tuần và cờ đánh dấu cuối tuần.

4. **Xác định sự kiện và thước đo (Identify Facts & Metrics):** Bảng sự kiện `fact_order_items` được xây dựng chứa các khóa ngoại trỏ đến các chiều và các chỉ số định lượng có tính chất cộng dồn (Additive) hoặc bán cộng dồn (Semi-additive):

   - `price`: Đơn giá sản phẩm (Additive).
   - `freight_value`: Cước phí vận chuyển từng món (Additive).
   - `total_item_value`: Tổng giá trị thanh toán cho dòng hàng (= `price`+`freight_value`).
   - `delivery_delay_days`: Số ngày giao hàng trễ so với dự kiến (Đơn vị: ngày).
   - `review_score`: Điểm đánh giá mức độ hài lòng của khách hàng (Thang điểm 1 đến 5 sao).

1.2 Cấu trúc Mô hình Ngôi sao (Star Schema)

Hệ thống sử dụng mô hình Ngôi sao thuần túy (*Pure Star Schema*). Khác với Snowflake Schema (chuẩn hóa các chiều), Star Schema phi chuẩn hóa (Denormalize) các chiều để giảm thiểu số lượng phép kết nối bảng (`JOIN`), giúp tối ưu hóa tối đa tốc độ truy vấn phân tích tổng hợp (OLAP).



Hình 1: Sơ đồ thực thể liên kết mô hình Ngôi sao (Star Schema) của Kho dữ liệu Olist

1.2.1 Khóa đại diện (Surrogate Keys)

Để đảm bảo tính độc lập giữa Kho dữ liệu và hệ thống giao dịch nguồn (OLTP), toàn bộ các bảng chiều và bảng sự kiện đều được gán **Khóa đại diện (Surrogate Key)** kiểu số nguyên (`INTEGER`):

- Khóa tự nhiên từ nguồn (như UUID `customer_id`, `product_id`) có kích thước lớn và tốn chi phí khi kết nối bảng. Việc chuyển sang khóa đại diện nguyên tuần tự 4-byte (`customer_key`, `product_key`, `seller_key`, `order_item_key`) giúp giảm 70% kích thước chỉ mục và tăng tốc độ xử lý `JOIN`.
- Riêng chiều thời gian `dim_date`, khóa đại diện được mã hóa dưới dạng số nguyên thông minh định dạng `YYYYMMDD` (ví dụ: ngày 28/10/2026 sẽ có `date_key = 20261028`), cho phép truy vấn lọc phạm vi ngày nhanh chóng mà không cần ép kiểu ngày tháng.

1.2.2 Chiều đóng vai trò chuyển vùng (Role-Playing Dimensions)

Trong bảng sự kiện `fact_order_items`, một đơn hàng trải qua 3 mốc thời gian quan trọng. Thay vì tạo ra 3 bảng chiều thời gian trùng lặp, mô hình chỉ lưu trữ duy nhất một bảng

dim_date và cho phép bảng Fact tham chiếu đến 3 lần với 3 vai trò khác nhau (Role-playing):

$$\begin{cases} \texttt{purchase_date_key} \longrightarrow \texttt{dim_date.date_key} & \text{(Thời điểm khách hàng chốt đơn)} \\ \texttt{delivered_date_key} \longrightarrow \texttt{dim_date.date_key} & \text{(Thời điểm đơn vị vận chuyển giao thành công)} \\ \texttt{estimated_date_key} \longrightarrow \texttt{dim_date.date_key} & \text{(Hạn chót cam kết giao hàng)} \end{cases} \tag{1}$$

1.2.3 Chiều thoái hóa (Degenerate Dimensions)

Các trường định danh nghiệp vụ như mã đơn hàng order_id, số thứ tự món hàng order_item_id và trạng thái đơn hàng order_status không cần thiết phải tách thành bảng riêng vì không có thuộc tính mô tả kèm theo. Chúng được giữ lại trực tiếp trên bảng Fact đóng vai trò là *Degenerate Dimensions*, phục vụ việc truy vết ngược về hệ thống nguồn khi cần thiết.

—

2 Quy trình Trích xuất, Biến đổi và Nạp Dữ liệu (ETL Pipeline)

2.1 Kiến trúc Tổng thể Quy trình ETL

Quy trình ETL được xây dựng hoàn toàn bằng ngôn ngữ Python kết hợp khung tính toán phân tán **Apache Spark (PySpark 4.2)**. Toàn bộ mã nguồn ETL được tổ chức theo cấu trúc mô-đun hóa cao trong thư mục etl/:

- etl/fetch_data.py: Quản lý dữ liệu nguồn, tự động đồng bộ từ Kaggle qua API kagglehub.
- etl/env.py: Khởi tạo môi trường độc lập (Portable Java 17, Hadoop Winutils, cấu hình Log).
- etl/extract.py: Khởi tạo SparkSession tối ưu và đọc 9 tập tin CSV với cấu hình xử lý đặc biệt.
- etl/transform.py: Lõi xử lý dữ liệu, chuẩn hóa địa lý, sinh khóa surrogate, tính toán các chỉ số kinh doanh.
- etl/load.py: Ghi dữ liệu ra hệ thống lưu trữ phân vùng định dạng Apache Parquet.

2.2 Giai đoạn Trích xuất (Extract)

Dữ liệu thô gồm 9 tập tin CSV với hơn 100.000 đơn hàng. Quá trình trích xuất đối mặt với các thách thức lớn về tính toàn vẹn dữ liệu:

- **Xử lý ký tự ngắt dòng trong văn bản đánh giá:** Trong tập tin olist_order_reviews_datase trường bình luận của khách hàng (review_comment_message) chứa nhiều ký tự xuống dòng thô (\n, \r). Nếu sử dụng trình đọc CSV tiêu chuẩn, Spark sẽ hiểu nhầm các ký tự này là kết thúc dòng, dẫn đến xáo trộn toàn bộ các cột phía sau (cột thời gian bị đẩy sang cột điểm số, gây lỗi ép kiểu NumberFormatException). Nhóm đã cấu hình trình đọc Spark với tùy chọn multiLine=True:

```
spark.read.csv(
    "dataset/olist_order_reviews_dataset.csv",
    header=True,
    inferSchema=True,
    multiLine=True
)
```

- **Ép kiểu an toàn (*Safe Casting*):** Sử dụng `try_cast` cho các trường số liệu để ngăn chặn ứng dụng dừng đột ngột khi gặp các giá trị dị biệt (anomalies).

2.3 Giai đoạn Biến đổi (Transform)

Giai đoạn biến đổi thực hiện việc làm sạch, làm giàu dữ liệu và mô hình hóa thành Star Schema:

2.3.1 Gom cụm và tổng hợp dữ liệu Geolocation

Tập dữ liệu địa lý `olist_geolocation_dataset.csv` chứa hơn 1.000.000 bản ghi tọa độ, trong đó một mã vùng (`zip_code_prefix`) bị trùng lặp hàng trăm lần với các sai số GPS nhỏ do thiết bị quét. Nếu đem bảng này kết nối trực tiếp với Khách hàng và Người bán sẽ gây ra hiện tượng bùng nổ tích Descartes (*Cartesian Product explosion*), làm phình to kích thước kho dữ liệu lên gấp 10 lần. Nhóm đã triển khai thuật toán tổng hợp tọa độ đại diện trung bình bằng PySpark:

$$\overline{\text{lat}} = \frac{1}{N}\sum_{i=1}^{N} \text{lat}_i, \quad \overline{\text{lng}} = \frac{1}{N}\sum_{i=1}^{N} \text{lng}_i \quad \text{(theo từng } \texttt{zip_code_prefix}\text{)} \tag{2}$$

Thuật toán này thu nhỏ bảng từ hơn 1.000.000 dòng xuống còn khoảng 19.000 dòng duy nhất, tăng tốc độ kết nối bảng lên gấp 15 lần.

2.3.2 Chuẩn hóa Ngôn ngữ và Làm sạch Dữ liệu

- Nối bảng dịch thuật `product_category_name_translation` với bảng sản phẩm để bổ sung tên ngành hàng chuẩn tiếng Anh (`product_category_name_english`). Với các sản phẩm không có danh mục hoặc bị thiếu, hệ thống tự động gán nhãn `"unknown"`.
- Sửa lỗi tên thuộc tính dữ liệu từ nguồn gốc (ví dụ: `product_name_lenght` viết sai chính tả trong CSV gốc được sửa thành chuẩn tiếng Anh `product_name_length`).

2.3.3 Sinh Khóa đại diện Toàn cục và Tính toán Chỉ số

Để sinh khóa tuần tự liên tục từ 1 $\rightarrow N$ cho các chiều, nhóm áp dụng hàm cửa sổ không phân vùng (*Global Window*):

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

window_spec = Window.orderBy("customer_id")
dim_customer = customers_df.withColumn(
```

```
    "customer_key", row_number().over(window_spec)
)
```

Trên bảng sự kiện `fact_order_items`, các chỉ số dẫn xuất quan trọng được tính toán trước:

$$\texttt{total_item_value} = \texttt{price} + \texttt{freight_value} \tag{3}$$

$$\texttt{delivery_delay_days} = \text{datediff}(\texttt{order_delivered_customer_date}, \texttt{order_estimated_delivery_da} \tag{4}$$

Nếu `delivery_delay_days` > 0, đơn hàng được xác định là giao trễ; nếu ≤ 0, đơn hàng được giao đúng hạn hoặc sớm hơn cam kết.

2.4 Giai đoạn Nạp (Load) và Tối ưu hóa Lưu trữ Parquet

Toàn bộ các bảng sau khi biến đổi được nạp vào thư mục $\texttt{data}_{w}arehouse/diØnhdng * *ApacheParquet * *.ParquetlØnhdnglutrdngct(ColumnarStorage)mangliccuthvttri$:

Tỷ lệ nén cao (High Compression Ratio): Giảm hơn 75% dung lượng lưu trữ đĩa so với CSV ban đầu nhờ thuật toán nén Snappy và nén từ điển (Dictionary Encoding).

Đọc cột chọn lọc (Column Projection): Khi một câu truy vấn OLAP chỉ cần tính tổng doanh thu (price), hệ thống chỉ đọc đúng cột đó trên đĩa mà bỏ qua hoàn toàn các cột khác, giảm thiểu triệt để I/O đĩa cứng.

Bảo toàn lược đồ dữ liệu (Self-describing Schema): Định dạng Parquet lưu sẵn siêu dữ liệu (Metadata) về kiểu dữ liệu (INT, DOUBLE, TIMESTAMP), giúp các động cơ truy vấn như DuckDB nhận diện kiểu dữ liệu tức thì mà không cần quét lại dữ liệu để suy đoán (*Zero Schema-Inference Overhead*).

—

3 Tầng Truy vấn Phân tích OLAP và Dịch vụ Backend API

3.1 Kiến trúc Động cơ Phân tích DuckDB In-Memory

Thay vì sử dụng các cơ sở dữ liệu quan hệ truyền thống (RDBMS) thường gặp nút thắt cổ chai khi thực thi các phép toán quét bảng lớn (*Full Table Scan*) hoặc tốn tài nguyên quản lý cụm như Spark, nhóm lựa chọn tích hợp **DuckDB** làm động cơ truy vấn OLAP cốt lõi cho Backend:

- DuckDB chạy nhúng trực tiếp trong cùng tiến trình ứng dụng (In-Process C++ Engine), triệt tiêu hoàn toàn độ trễ truyền tải mạng (Network IPC overhead).
- DuckDB tự động nhận diện và truy vấn trực tiếp trên các tập tin Parquet thông qua kỹ thuật *Predicate Pushdown* (đẩy điều kiện lọc xuống thẳng tầng đọc đĩa).
- Kết quả tổng hợp đa chiều được tính toán ở tầng nhị phân với kiến trúc thực thi vector hóa (*Vectorized Query Execution*), phản hồi các phép tính tổng hợp trên hàng trăm nghìn dòng dữ liệu trong thời gian dưới 50 ms.

3.2 Hiện thực các Thao tác Phân tích Đa chiều (OLAP Operations)

Backend được xây dựng bằng framework **FastAPI**, cung cấp hệ thống RESTful APIs phục vụ trực tiếp cho giao diện Dashboard và các thao tác phân tích dữ liệu đa chiều:

Bảng 1: Bảng ánh xạ các thao tác phân tích OLAP với RESTful API Endpoints

Thao tác OLAP	**Ý nghĩa Nghiệp vụ**	**API Endpoint Backend**
Slice (Cắt lát)	Cố định một chiều phân tích (ví dụ: chỉ phân tích các chỉ số trong Năm 2018).	`GET /api/olap/kpis?year=2018`
Dice (Xắt hạt lựu)	Lọc đồng thời trên nhiều chiều không gian (ví dụ: Năm 2018, khách hàng tại bang `SP`, danh mục `health_beauty`).	`GET /api/olap/kpis?year=2018&state=SP&catego`
Roll-up (Cuộn gộp)	Tổng hợp doanh thu và số lượng đơn hàng từ cấp độ tháng/quý lên cấp độ năm để xem xu hướng dài hạn.	`GET /api/olap/sales-trend?granularity=year`
Drill-down (Khoan sâu)	Đi sâu từ cấp độ năm xuống từng quý và từng tháng chi tiết để giải thích nguyên nhân tăng giảm doanh số.	`GET /api/olap/sales-trend?granularity=month`
Top-N Analysis	Xếp hạng 10 danh mục sản phẩm có doanh thu hoặc số lượng bán cao nhất toàn sàn.	`GET /api/olap/top-categories?limit=10`
Geo-Distribution	Phân bổ doanh số và đơn hàng trên 27 bang của Brazil để đánh giá thị trường trọng điểm.	`GET /api/olap/geo-distribution`
Ad-hoc OLAP SQL	Cho phép người dùng nhập trực tiếp câu lệnh SQL tổng hợp đa chiều tùy biến từ bảng điều khiển console.	`POST /api/olap/execute-query`

3.3 Đánh giá Hiệu năng Hệ thống

Quá trình thử nghiệm thực tế trên máy trạm cấu hình phổ thông (Intel Core i5, 16GB RAM) cho thấy hiệu năng vượt trội:

- Thời gian thực thi toàn bộ quy trình ETL tự động cho 100.000 đơn hàng: ≈ 45 **giây**.
- Dung lượng toàn bộ Kho dữ liệu sau khi nén Parquet: ≈ 18 **MB** (so với hơn 120 MB của 9 tập tin CSV thô ban đầu).
- Độ trễ trung bình của các API phân tích OLAP (KPIs, Sales Trend, Top-N): $15 - 35$ **ms**, đảm bảo trải nghiệm tương tác thời gian thực mượt mà trên giao diện người dùng.